COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 8, 2026
NAME:	Baltazar, Kenji Formilleza, Huone Anthony	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **[Basic Algorithm Analysis]**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

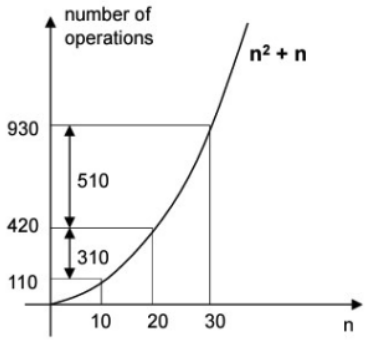
Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:
Let m = the length of array x.
Let n = the length of array y.
The loop in diff repeats n times.
Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **n^2**

- Assume that m = n. (The arrays are the same size)
  Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
(Baltazar)
For the worse case scenario, since the loop runs in diff without any known limiting factor that would make m x n, and assuming that m = n, it will become n x n which would make n^2.
For the graph, at n = 10, it has 110 operations, at n = 20, it has 420 operations, whilst at n = 30, it has 930 operations, so it is clearly exponentiating and increasing per n size. Due to this fact, the graph does not have a straight line going up but curves up like a half parabola.

Analysis:
(Formilleza)
The graph shown is curved upward because the work grows like n2, The loop runs n times and each time it does a search that looks through n items. At n = 10 it takes 110 steps, at 20 it takes 420, and at 30 it takes 930, after each jump are 310 then 510, so the bigger the array the bigger the number of operations. The extra n from the loop on top is small compared to n squared when n gets big so we ignore it which makes the worst case scenario O(n^2).

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

CODE

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;
    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }
        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated;}

        // Two people are same if they have the same name
```

```
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }
        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }
        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
    };

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true){
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
        return true;
            else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
```

```
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}

void search_test(int size, Student p) {
    std::vector<Student> people;

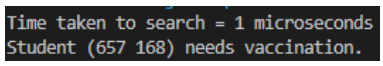
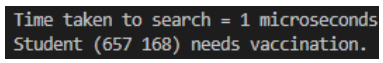
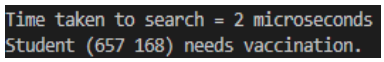
    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
    << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
    << ") " << "does not need vaccination." << std::endl;
}


int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	1 microsecond		The search takes in 1 microsecond. It only needs about 10 steps.
10000	1 microsecond		Even though the size multiplied by 10, the search takes around 1 microsecond as well, and it only takes about 14 steps.
100000	2 microsecond		As the size grew 100 more times, the search took 2 microseconds greater than the previous 2, taking around 17 steps.

Analysis:
From what we observed in the given output, the execution speed really didn't increase significantly even though the input sizes increased by tens each time. We concur that this is due to the fact that this is a binary search, meaning that it is accompanied by a time complexity of O(log n), meaning that the input sizes are being halved which significantly reduces the steps when we compare it to a linear search. Overall, the experiment shows that the time stayed almost the same even though the input sizes grew increasingly large which aligns with the theory regarding the code and the concept of binary search.

2. SUPPLEMENTARY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

bool algo_uniqueA(const std::vector<int>& A){
    int n = A.size();
    for(int i = 0; i < n - 1; i++){
        for(int j = i + 1; j < n; j++){
            if(A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

void search_test(int size) {
    std::vector<int> A;

    for(auto i = 0; i < size; i++){
        A.push_back(i);
    }

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
    bool search_result = algo_uniqueA(A);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if(search_result)
        std::cout << "Array of size " << size << " is unique." << std::endl;
    else
        std::cout << "Array of size " << size << " is not unique." << std::endl;
```

```
41  }
42
43  int main(){
44      search_test(1000);
45      search_test(12000);
46      search_test(26700);
47      return 0;
48  }
```

OUTPUT

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	4890 microsecond	Time taken to search = 4890 microseconds Array of size 1000 is unique.	Although the array size is only 1000, the time taken took too long compared to the procedure. This is due to the time complexity of $O(n^2)$.
12000	491949 microsecond	Time taken to search = 491949 microseconds Array of size 12000 is unique.	Now that the input size increases, we can observe that the execution speed was greatly affected thus increasing significantly.
26700	2487383 microsecond	Time taken to search = 2487383 microseconds Array of size 26700 is unique.	When this was executed, there was an obvious delay before the text was printed. Showing that once the input size is too big, $O(n^2)$ can affect the execution to delay and also a possible crash when the input size is too large.

Analysis:The first code checks every pair of elements using two nested loops, so it is $O(n^2)$. Since all the values are unique, it never returns early and always runs the full worst case. The output shows this, because going from 1000 to 12000 is 12 times bigger and the time went up about 100 times, which is close to 12 squared. Sorting first or using a hash set would be much faster.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

int rpower(int x, int n){
    if(n == 0) return 1;
    else return x * rpower(x, n - 1);
}

int brpower(int x, int n){
    int y;
    if(n == 0) return 1;
    if(n % 2 == 1){
        y = brpower(x, (n - 1)/2);
        return x * y * y;
    }
    else{
```

```
        y = brpower(x, n/2);
        return y * y;
    }
}

void power_test(int n){
    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
    int searchr_result = rpower(1, n);

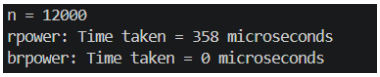
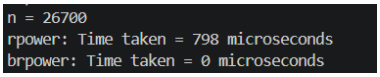
    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "n = " << n << std::endl;
    std::cout << "rpower: Time taken = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;

    begin = std::chrono::steady_clock::now();
    int searchbr_result = brpower(1, n);
    end = std::chrono::steady_clock::now();
    std::cout << "brpower: Time taken = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
}

int main() {
    power_test(1000);
    power_test(12000);
    power_test(26700);
    return 0;
}
```

OUTPUT

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	rpower: 26 ms Brpower: 0 ms	n = 1000 rpower: Time taken = 26 microseconds brpower: Time taken = 0 microseconds	From just 1000 input size, we can see the difference between rpower and bpower. Where rpower took 26ms to finish due to it being O(n) where it needs to multiply one step per time. Meanwhile, brpower took 0 ms due to it being O(log n) where it keeps dividing itself by 2 thus reducing the time

			significantly.
12000	rpower: 358 ms Brpower: 0 ms		When the input size increased, we can see a significant increase in delay in execution for rpower. From this observation, we can concur that rpower multiplies greatly depending on how high the input size is. However, we can see that there is low to zero difference in brpower as O(log n) benefits when the input sizes increases compared to rpower. This is because brpower and the time complexity of O(log n) are best suited for big inputs compared to rpower.
26700	rpower: 798 ms brpower: 0 ms		As for extreme cases, we can still see a significant difference for rpower and brpower which is increasing the more input size increases.

Analysis: rpower multiplies one step at a time, so it makes n calls and is O(n) brpower uses repeated squaring and cuts the power in half every call, so it is O of log n. The output shows rpower time growing as n grows, from 26 to 798 microseconds, while brpower stays at 0 microseconds because it only needs around 15 calls, which is too fast to measure. So brpower is way faster for big n.

3. CONCLUSION

In conclusion we learned how to measure how fast an algorithm is using both theory and experiment.
inear search grows with n while diff grows with n squared, and in our experiment binary search stayed very fast even at 100,000 items, which matches O of log n. For supplementary Unique is O of n squared in the worst case, rpower is O of n, and brpower is O of log n, and our timing results followed the theory. We also saw that rpower can crash with a stack overflow when n is too big.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.